ReqShift

Requirements prose in, a proved goal model out.

What it is

ReqShift reads an engineering requirements document written in ordinary language and turns it into a model a solver can reason about. It then reports what the solver proved about that model.

It is built on top of GoalShift, the analysis tool behind the thesis. GoalShift does the proving. ReqShift is the front end that gets a real document into a form GoalShift can accept, which until now had to be done by hand.

The problem it addresses

Modern machines are rarely one machine. A line has a variable number of robots. A warehouse has a variable number of shuttles. A charging site has a variable number of chargers and vehicles.

Requirements documents almost never say how many. They describe what each part must do, and they leave the question of how many of each part will actually be present to be settled later, on site. Nobody checks whether the system still works at every combination the document allows.

> A study of 31 real requirements documents across four industries found the same thing every time. The documents state the behaviour. They do not state the composition. The gap is not an oversight in one document. It is how these documents are written.

What it does, in four steps

1. Read. An AI reads the document and proposes a goal model. Every element it proposes carries the sentence it came from.

↓

2. Report the gaps. The tool lists what the document never stated, above all the quantities. This costs nothing to produce and is checkable against the document itself.

↓

3. Take the numbers. The engineer fills in the quantities that were missing. These are usually known, just never written down.

↓

4. Prove. The solver checks every combination the numbers allow, and reports which ones work, which ones break, and why.

What comes out

The solver does not return an opinion. It returns facts about the model, and each one traces back to a sentence in the document.

- **Behaviour classes.** Out of all the combinations the document permits, only a handful behave differently from each other. The tool finds that handful. Twenty four possible configurations often collapse to four distinct behaviours.
- **Broken configurations.** The exact combinations in which some part of the system cannot meet its goal, named one by one.
- **Numbers that do nothing.** Upper limits written into the document that cannot change any

outcome. These can be shown to be decoration rather than guessed at.

- **Real ceilings.** Where one part limits another. For example a fleet capped at seven by the capacity of what serves it, not by its own stated maximum of twelve.

Why the answers can be trusted

This is the part that matters most, so it is worth being precise about it.

A language model cannot tell you when it is wrong. A solver cannot read prose. Their weaknesses are opposites, so each one covers the other. ReqShift uses each for only the job it is good at.

> **The AI produces the input. The solver produces the answer.**
> The AI writes a draft goal model, which a person reviews. Every finding after that point is computed arithmetically from the solver's output, by code that never calls a language model at all. A finding cannot be invented, because nothing that invents things is involved in producing it.

The draft is meant to be reviewed, not trusted. That is why every element the AI proposes is shown next to the sentence it was taken from, and why the tool separates what the document actually stated from what the AI assumed.

What it does not do

- It does not check that the requirements are the right requirements. It checks that they hold together, and that the system works at every permitted size.
- It does not replace the engineer's review of the model. The AI drafts, a person confirms.
- It cannot find something the document never mentioned. If a whole part of the system is missing from the text, it is missing from the model.
- It applies to systems with a variable number of parts. For a single fixed machine there is nothing for it to say.

Where it stands today

The analysis engine is the one described in the thesis and it is finished. It has been run end to end on real requirements documents from four different industries, and on a hand built case study which it reproduced from the text alone.

There are two engines behind it. The default is the SAT engine from the thesis, whose results are proved by the published encoding. A second, experimental engine based on z3 handles larger quantities and can also take timing and capacity into account. The two agree on every model both can express, checked over seventy seven generated test models with no disagreement.

GoalShift is the analysis tool described in the thesis and is not modified by ReqShift. It is imported unchanged, so the artifact the research describes stays exactly as it was.